

Python

import os

import zipfile

Define content for CLAUDE.md

claude_md_content = """# CLAUDE.md - 스마트카드 연동 웹 서비스 개발 명세서 (최소 구성)

본 문서는 로컬 환경에서 Docker 서버와 호스트 PC 클라이언트(네이티브 브리지) 간의 스마트카드 통신 기능을 검증하기 위한 최소 기능 프로토타입 개발 명세서입니다.

1. 프로젝트 아키텍처 및 디렉토리 구조

1.1 시스템 구성도

코드 출력

CLAUDE.md 파일을 성공적으로 생성했습니다.

[Docker 컨테이너 (SWebserver)]

| ▲ (1) 웹페이지 요청 / API 통신 (HTTP/HTTPS)

▼ |

[웹 브라우저 (Client UI)]

| ▲ (2) APDU 명령어 전송 / 응답 수신 (WebSocket)

▼ |

[로컬 호스트 OS (SClient 에이전트)]

| ▲ (3) 스마트카드 제어 (PC/SC 스택)

▼ |

[스마트카드 리더기 & 스마트카드]

1.2 디렉토리 구조 (SWT/)

SWT/

- |— CLAUDE.md # 본 개발 명세서
- |— SWebserver/ # 서버 역할 (Docker 구성 및 웹 서버 소스)
 - | |— Dockerfile
 - | |— docker-compose.yml
 - | |— server.js # 웹페이지 및 최소 API 제공
- |— SClient/ # 클라이언트 역할 (로컬 네이티브 브리지)
 - | |— package.json
 - | |— bridge.js # PC/SC - WebSocket 중계 스크립트
- |— doc/ # 프로젝트 관련 문서 및 참고 자료 저장용

2. 요구사항

2. 컴포넌트별 최소 구현 요구사항

2.1 SWebserver (Docker 서버 환경)

****목적:**** 브라우저에 UI 화면을 제공하고 향후 백엔드 로직(명령어 생성 및 검증) 확장
준비

****기본 기술:**** Node.js (Express) 또는 단순 Nginx + Static HTML, Docker

****제공 화면 (UI 요건):****

****카드 리셋 버튼 ('Card Reset'):** 카드를 초기화하고 ATR(Answer To Reset) 값을
확인하기 위한 트리거

****APDU 입력창 (Text Input):**** 송신할 APDU 명령어(예: "00A40400...")를 hex(Hex)
문자열로 입력하는 창

****APDU 전송 버튼 ('Send APDU'):** 입력된 APDU를 브리지로 전송하는 트리거

****응답창 (Textarea / Readonly):**** 카드로부터 리턴된 응답 APDU 및 상태 워드(Status)

Word, 예: '9000')를 표시하는 영역

2.2 SClient (로컬 PC 네이티브 브리지)

****목적:**** 브라우저의 웹소켓 요청을 받아 OS의 PC/SC 드라이버를 통해

하드웨어(스마트카드)와 직접 통신

****기반 기술:**** 호스트 OS에서 구동되는 Node.js (라이브러리: 'ws', '@pokusew/pcsc-lite

등)

****핵심 기능:****

* 'ws://localhost:8081' 포트로 웹소켓 서버 오픈

* 브라우저의 '카드 리셋' 요청 수신 시, 연결된 리더기의 카드를 연결(Connect)하고 ATR

정보를 획득하여 브라우저로 전송

* 브라우저의 'APDU 전송' 요청 수신 시, hexa 문자열을 바이너리 버퍼로 변환 후 카드에

전송(Transmit), 응답 버퍼를 다시 hexa 문자열로 변환하여 브라우저로 회신

■

3. 프로토타입 통신 시나리오 (데이터 흐름)

1. ****초기화:**** 사용자가 스마트카드 리더기를 USB에 연결하고, 로컬 PC에서

SClient/bridge.js'를 실행합니다

2. ****웹 접속:**** 브라우저를 통해 Docker 서버('http://localhost:8080')에 접속하여 UI

페이지를 로드합니다

3. ****웹소켓 연결:**** 로드된 웹페이지의 JavaScript가 로컬 브리지('ws://localhost:8081')와

웹소켓 커넥션을 맺습니다

4. ****카드 리셋 테스트:**** * 사용자가 '카드 리셋' 버튼 클릭 → 브라우저가 웹소켓으로

{ "action": "RESET" } 전송

* 브리지 에이전트가 하드웨어 리셋 수행 → 획득한 ATR 값을 브라우저로 회신 → 웹 화면

응답창에 ATR 출력

5. ****APDU 전송 테스트:****

* 사용자가 입력창에 APDU 명령어 입력 후 'APDU 전송' 버튼 클릭 → 브라우저가

웹소켓으로 { "action": "TRANSMIT", "apdu": "00A4040000" } 전송

```
* 브리지 에이전트가 카드에 명령 전달 → 카드 응답(Data + SW) 수신 → 브라우저로 회신  
→ 웹 화면 응답창에 결과 출력
```

```
■
```

```
## 4. 빌드 및 실행 가이드 (Next Step)
```

```
### 4.1 서버 실행 (Docker)
```

```
bash
```

```
cd SWebserve
```

```
docker-compose up --build
```

- 이후 브라우저에서 `http://localhost:8080`으로 접속

4.2 클라이언트 브리지 실행 (로컬 호스트 OS)

```
Bash
```

```
cd Client
```

```
npm install
```

```
code bridge.js
```

5. 개발 주의사항 및 팁

- **CORS 및 보안:** 로컬 웹소켓(`ws://localhost`) 통신은 브라우저의 당해 Origin 정책(Same-Origin)에서 비교적 자유로우나, 운영 환경으로 전환 시 HTTPS 환경에서는 안전한 웹소켓(`wss://`)만 허용되므로 초기 설계 시 주소 확장을 고려해야 합니다.
- **하드웨어 드라이버:** 로컬 PC에 스마트카드 리더기 제조사의 CCID 드라이버가 올바르게 설치되어 OS 수준에서 리더기를 인식하고 있어야 합니다. (윈도우의 경우 '스마트 카드' 서비스가 시작 상태여야 함)

```
*****
```

Create the directory structure locally to

package it for the user

```
base_dir = "SWT"
sub_dirs = ["SWebserver", "SClient", "doc"]
for s_dir in sub_dirs:
    os.makedirs(os.path.join(base_dir, s_dir), exist_ok=True)
```

Write CLAUDE.md

```
with open(os.path.join(base_dir, "CLAUDE.md"), "w", encoding="utf-8") as f:
    f.write(claude_md_content)
```

Package into a zip file for easy download

```
zip_filename = "SWT_Project_Structure.zip"
with zipfile.ZipFile(zip_filename, 'w', zipfile.ZIP_DEFLATED) as zipf:
    for root, dirs, files in os.walk(base_dir):
        for file in files:
            file_path = os.path.join(root, file)
            zipf.write(file_path, os.path.relpath(file_path, os.path.dirname(base_dir)))
print("Project structure created and zipped successfully.")
```

요청하신 폴더 구조와 개발 명세서 'CLAUDE.md' 파일 생성이 완료되었습니다.

개발 편의성을 위해 명세서뿐만 아니라 요청하신 전체 폴더 구조(SWT/SWebserver, SWT/SClient, SWT/doc)까지 포함된 압축 파일(ZIP)로 준비해 드립니다.

file-tag: code-generated-file-0-1780968832846904047



작성된 **CLAUDE.md** 개발 명세서의 핵심 내용 요약은 다음과 같습니다.

📌 최소 요구 기능 설계 요약

1. **SWebserver (Docker 영역)

- * 역할: 클라이언트(브라우저)에게 화면을 제공하는 HTTP 웹 서버

- * UI 구성

* `카드 리셋(Card Reset)` 버튼

* `APDU 입력창` (명령어를 Hex 문자열로 입력)

* `APDU 전송(Send APDU)` 버튼

* `응답창` (명령 결과 및 Status Word 출력 영역)

2. ****SClient (로컬 PC 네이티브 브리지 영역)****

* 역할: 호스트 운영체제(로컬 PC)에서 돌아가면서, 브라우저와

웹소켓('ws://127.0.0.1')으로 통신하고 물리적 USB 스마트카드 리더기를 제어하는 Node.js

기반 브리지 서버

다운로드하셔서 로컬 PC에 압축을 푸신 후, 각 폴더(SWebserver, SClient)에 실제 코드를

작성해 나가시면 됩니다.

기본적인 아키텍처와 명세서가 완성되었으니, 다음 단계로 ****SWebserver에 들어갈**

화면(HTML/JS) 파일**을 작성하거나, ****SClient에 들어갈 로컬 브리지(Node.js) 스크립트****

작성을 시작하시려면 언제든지 말씀해 주세요.